
Smooth Component Filtering (SCF)

Release 1.0.0

AE,SB,BB

Jun 01, 2026

Contents:

1	scf module	1
1.1	Smooth Component Filtering (SCF)	1
1.2	Window functions	2
1.3	Module functions:	3

SCF MODULE

1.1 Smooth Component Filtering (SCF)

By **convolved gridded visibility** $\mathcal{V}_{cg}(\nu_n)$ and given a **suitable window function** $H(n)$ of **specified smoothing scale** (controlled by parameter N), we calculate the following -

1. $(\mathcal{V}_{cg} * H)(\nu_n) = \sum_m \mathcal{V}_{cg}(\nu_m) H(n - m)$ and,
2. $(U * H)(\nu_n) = \sum_m U(\nu_m) H(n - m)$

Dividing them we get :

$$\boxed{\mathcal{V}_{cg}^S(\nu_n) = (\mathcal{V}_{cg} * H)(\nu_n) / (U * H)(\nu_n)} \quad (1.1)$$

We refer $\mathcal{V}_{cg}^S(\nu_n)$ as **Smooth Component** of the convolved gridded visibility $\mathcal{V}_{cg}(\nu_n)$ and **subtracting** it from the $\mathcal{V}_{cg}(\nu_n)$ gives us the **filtered component**.

$$\boxed{\mathcal{V}_{cg}^F(\nu_n) = \mathcal{V}_{cg}(\nu_n) - \mathcal{V}_{cg}^S(\nu_n)} \quad (1.2)$$

$\mathcal{V}_{cg}(\nu_i) = 0$ for the channels ν_i , that are **flagged** and the division by $(U * H)(\nu_n)$ is necessary to **ensure correct normalization after smoothing**, i.e. $(\mathcal{V}_{cg} * H)(\nu_n)$. Here $U(\nu_i) = 0$ for the flagged channels and $U(\nu_i) = 1$ otherwise. The **smoothing is restricted to a smaller range near the boundaries** and to account for this N no of **channels are discarded** from both ends of $\mathcal{V}_{cg}^F(\nu_n)$ to account this.

If $\Delta\nu_c$ denotes the **channel separation** then for a given N corresponds to **smoothing scale** of $N\Delta\nu_c$. For example $N = 25, 50$ and 75 and $\Delta\nu_c = 0.04\text{MHz}$ corresponds to smoothing scales of $N\Delta\nu_c = 1, 2$ and 3 MHz respectively.

Notes

We expect **SCF** to **suppress** the values of $|P(k_\perp, k_\parallel)|$ somewhere between the half and full width of the filter, i.e.

$$[k_\parallel]_F / 2 \leq k_\parallel \leq [k_\parallel]_F$$

Where $[k_\parallel]_F = \frac{2\pi}{r' N \Delta\nu_c}$.

1.2 Window functions

This module supports **four types of window functions** namely -

1. **Hann/Hanning** window function $H_{\text{Hann}}(n)$.
2. **Hamming** window function $H_{\text{Hamming}}(n)$.
3. **Blackman** window function $H_{\text{Blackman}}(n)$.
4. **Kaiser** window function $H_{\text{Kaiser}}(n)$ with $\beta = 14$.

The functional forms are given as :

$$\begin{aligned}
 H_{\text{Hann}}(n) &= \frac{1}{2N} \left[0.5 + 0.5 \cos \left(2\pi \frac{n}{2N} \right) \right] & -N \leq n \leq N \\
 H_{\text{Hamming}}(n) &= \frac{1}{2N} \left[0.54 + 0.46 \cos \left(2\pi \frac{n}{2N} \right) \right] & -N \leq n \leq N \\
 H_{\text{Blackman}}(n) &= \frac{1}{2N} \left[0.42 + 0.5 \cos \left(2\pi \frac{n}{2N} \right) + 0.08 \cos \left(4\pi \frac{n}{2N} \right) \right] & -N \leq n \leq N \\
 H_{\text{Kaiser}}(n) &= \frac{1}{2N} \frac{I_0 \left(\sqrt{1 - \left(\frac{n}{N} \right)^2} \right)}{I_0(\beta)} & -N \leq n \leq N
 \end{aligned}$$

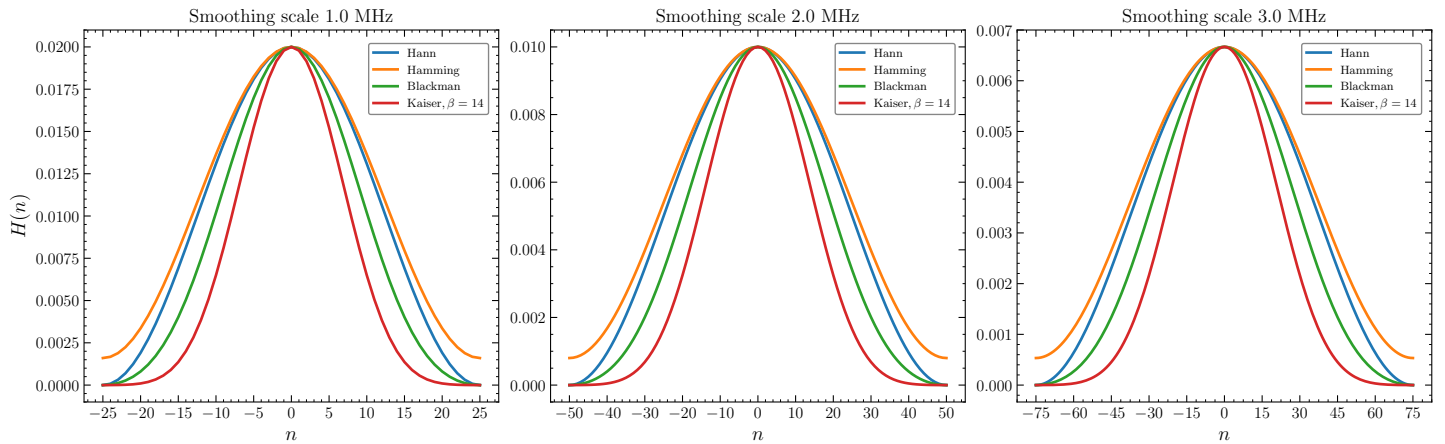


Fig. 1: Different window functions with different smoothing scales.

1.3 Module functions:

```
scf.doscf(GV, SM, window='hann', method='auto')
```

Given gridded visibility array it performs **SCF (Smooth Component Filtering)** given a **window function** and **suitable smoothing scale**. It **subtracts out the smooth part** of it and returns **residual filtered visibilities**.

Parameters

- `GV (np.ndarray)` – Convolved-gridded visibility array. The last dimension must have shape `(,768)`.
- `SM (int or float)` – **Smoothing scale** in MHz of specified window function.
- `window (str {'hann', 'hamming', 'blackman', 'kaiser'}, optional)` – Window function name. Valid ones are **hann**, **hamming**, **blackman**, **kaiser**. If not any of this it kills the current iteration. By default hann window.
- `method (str {'auto', 'direct', 'fft'}, optional)` – The **method** by which the **convolution** is performed. Either by **direct** or **fft**. By default is **auto**, means **whichever is faster is applied**.

Returns

GV_filtered – The **filtered visibility** which is obtained by **subtracting out the smooth part**. Shape same as input GV array **except in the last dimension** in which first and last $NW = SM/\Delta\nu_c$ no of channels are sliced. Where $\Delta\nu_c$ is the separation between two frequency channels in MHz. By default the value is *0.04*.

Return type

np.ndarray

Examples

```
>>> import numpy as np
>>> import myutils.scf.scf as sf
Imported myutils
Imported SCF,          Import Time : 30/05/26 | 05:30:25 PM
>>> GV = np.load("/home/cts23ph/git_package_myutils/tge/GV_7200.npy") # GV array
>>> print(f"{GV.shape = }")
GV.shape = (2, 457, 457, 768)
>>> SM = 2 # smoothing scale in MHz
>>> NW = int(SM/0.04)
>>> print(f"{NW = }")
NW = 50
>>> # perform SCF
>>> GVr = sf.doscf(GV, SM, window = 'hann', method = 'fft')
<===== Performing SCF =====>
<=== Start Time: 30/05/26 | 05:29:00 PM ===>
Convoluting kernel : fft
Window function    : Hanning
Smoothing scale    : 2 MHz
<=== End Time   : 30/05/26 | 05:29:09 PM ===>
```

(continues on next page)

(continued from previous page)

```

<=== Elapsed    : 00:00:09.24, 9.240 seconds.
<===== Done SCF =====>
>>> print(f"Gvr.shape = {Gvr.shape}") # 768-2*NW
Gvr.shape = (2, 457, 457, 668)

```

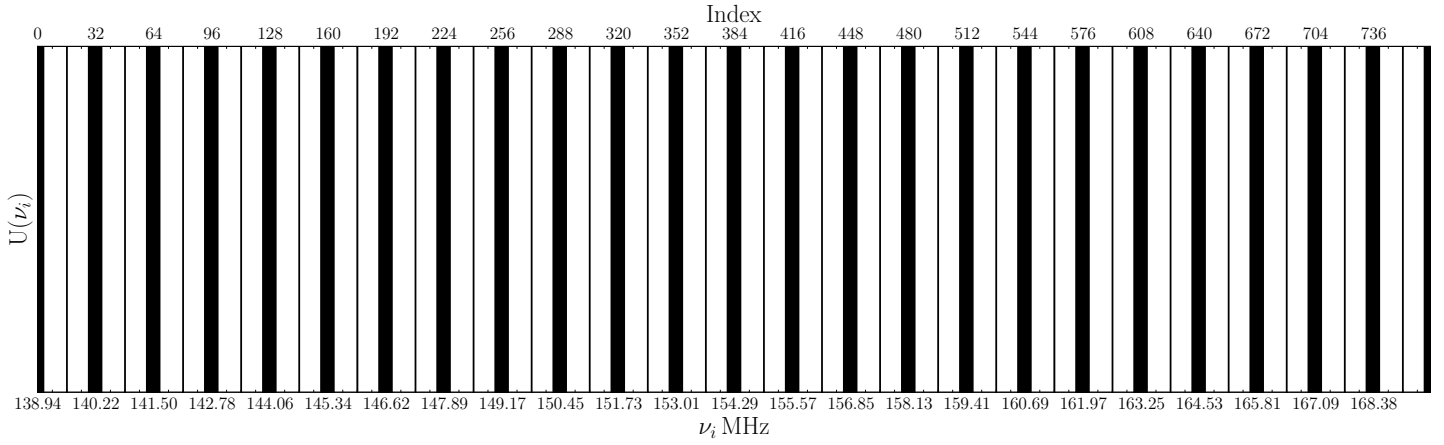


Fig. 2: The Flagging of MWA data. The black colored regions are flagged.

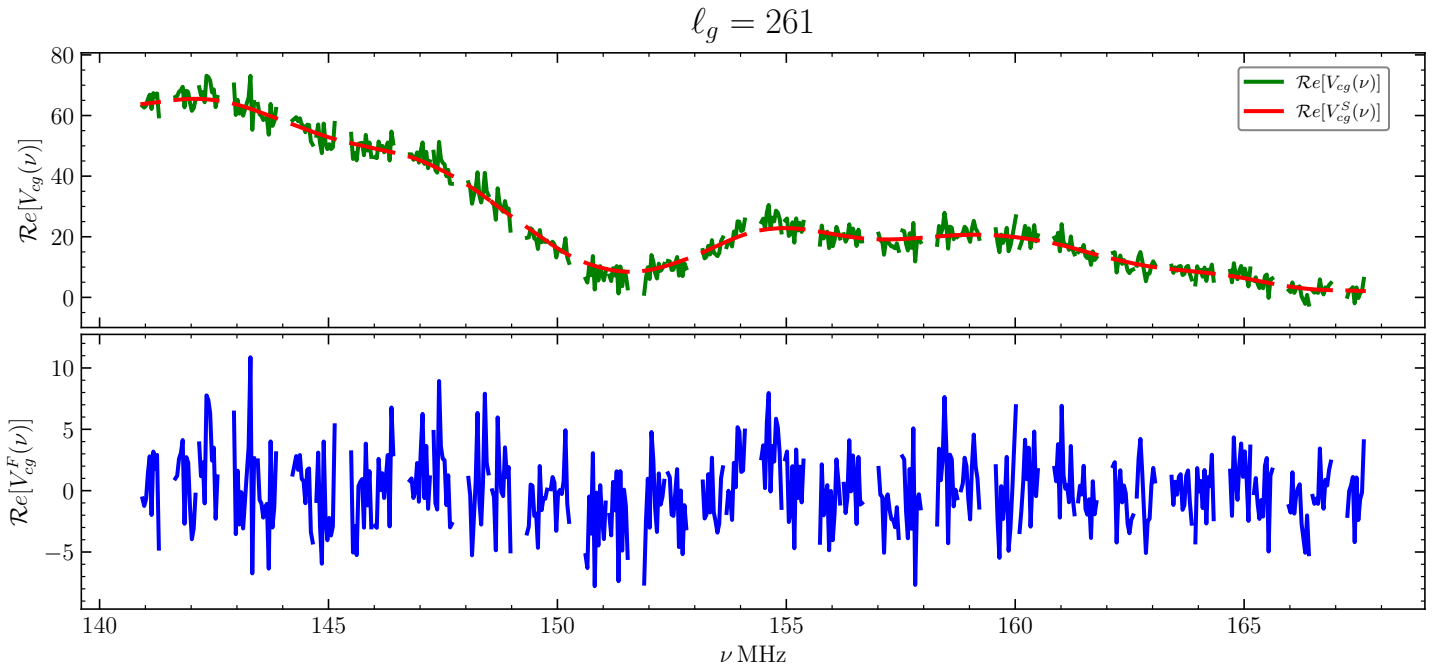


Fig. 3: Real part of $\mathcal{V}_{cg}$, $\mathcal{V}_{cg}^S$, $\mathcal{V}_{cg}^F$ as a function of frequency ν for a fixed grid point corresponding to $\ell_g = 261$. This is shown only for XX polarization.